

Building Your First Agent

Building a Simple AI Agent, Part 1

Now that you understand the agent loop and how to craft effective prompts, we can build a simple AI agent. This agent will be able to list files in a directory, read their content, and answer questions about them. We'll break down the agent loop—how it receives input, decides on actions, executes them, and updates its memory—step by step.

The Agent Loop in Python

The agent loop is the backbone of our AI agent, enabling it to perform tasks by combining response generation, action execution, and memory updates in an iterative process. This section focuses on how the agent loop works and its role in making the agent dynamic and adaptive.

- Construct Prompt:** Combine the agent's memory, user input, and system rules into a single prompt. This ensures the LLM has all the context it needs to decide on the next action, maintaining continuity across iterations.
- Generate Response:** Send the constructed prompt to the LLM and retrieve a response. This response will guide the agent's next step by providing instructions in a structured format.
- Parse Response:** Extract the intended action and its parameters from the LLM's output. The response must adhere to a predefined structure (e.g., JSON format) to ensure it can be interpreted correctly.
- Execute Action:** Use the extracted action and its parameters to perform the requested task with the appropriate tool. This could involve listing files, reading content, or printing a message.
- Convert Result to String:** Format the result of the executed action into a string. This allows the agent to store the result in its memory and provide clear feedback to the user or itself.
- Continue Loop:** Evaluate whether the loop should continue based on the current action and results. The loop may terminate if a "terminate" action is specified or if the agent has completed the task.

The agent iterates through this loop, refining its behavior and adapting its actions until it reaches a stopping condition. This process is what enables the agent to interact dynamically and respond intelligently to tasks.

Here's how these steps come together in code:

```
# The Agent Loop
while iterations < max_iterations:

    # 1. Construct prompt: Combine agent rules with memory
    prompt = agent_rules + memory

    # 2. Generate response from LLM
    print("Agent: thinking...")
    response = generate_response(prompt)
    print(f"Agent response: {response}")

    # 3. Parse response to determine action
    action = parse_action(response)

    result = "Action executed"

    if action["tool_name"] == "list_files":
        result = ("result":list_files())
    elif action["tool_name"] == "read_file":
        result = ("result":read_file(action["arg"]]["file_name"])
    elif action["tool_name"] == "error":
        result = ("error":action["arg"]]["message"])
    elif action["tool_name"] == "terminate":
        print(action["arg"]]["message"])
        break

    else:
        result = ("error":f"Unknown action: {action['tool_name']}")

    print(f"Action result: {result}")

    # 5. Update memory with response and results
    memory.extend([
        ("role": "assistant", "content": response),
        ("role": "user", "content": json.dumps(result))
    ])

    # 6. Check termination condition
    if action["tool_name"] == "terminate":
        break

    iterations += 1
```

Step 1: Constructing the Agent Prompt

The prompt is created by appending the agent's rules (system message) to the current memory of interactions. Part of the memory is a description of the task that the agent should perform. This ensures the agent is always aware of its tools and constraints while also remembering past actions.

```
prompt = agent_rules + memory
```

Explanation:

- agent_rules** : This contains the predefined system instructions, ensuring the agent behaves within its defined constraints and understands its tools.
- memory** : This is a record of all past interactions, including user input, the agent's responses, and the results of executed actions.

By constructing the prompt this way, the agent retains continuity across iterations, ensuring it can adapt its behavior based on previous actions and results. The memory tells it what just happened, what happened in the past, and informs its decision of the next action.

Agent Rules: Defining the Agent's Behavior

Before the agent begins its loop, it must have a clear set of rules that define its behavior, capabilities, and constraints. These **agent rules** are specified in the system message and play a critical role in ensuring the agent interacts predictably and within its defined boundaries.

How it works in code:

The `agent_rules` are written as a system message that instructs the LLM on how the agent should behave, what tools it has available, and how to format its responses. These rules are included at the start of the prompt for every iteration.

Agent Rules: Defining the Agent's Behavior

Before the agent begins its loop, it must have a clear set of rules that define its behavior, capabilities, and constraints. These **agent rules** are specified in the system message and play a critical role in ensuring the agent interacts predictably and within its defined boundaries.

```
agent_rules = [
    {"role": "system",
    "content": """
    You are an AI agent that can perform tasks by using available tools.

    Available tools:
    - list_files() -> list(str): List all files in the current directory.
    - read_file(file_name: str) -> str: Read the content of a file.
    - terminate(message: str): End the agent loop and print a summary to the user.

    If a user asks about files, list them before reading.

    Every response MUST have an action.
    Respond in this format:

    '''action
    {
        "tool_name": "insert_tool_name",
        "arg": { ...fill in any required arguments here... }
    }
    '''
    """
]
```

Explanation:

- Role of system messages:** The `system` role in the messages list is used to establish ground rules for the agent. This ensures the LLM understands what it can do and how it should behave throughout the session.
- Tools description:** The agent rules explicitly list the tools the agent can use, providing a structured interface for interaction with the environment.
- Output format:** The rules enforce a standardized output format (''' action (...) '''), which makes parsing and executing action easier and less error-prone.

Each of the "tools" in the system prompt correspond to a function in the code. The agent is going to choose what function to execute and when. Moreover, it is going to decide the parameters that are provided to the functions.

The agent is not creating the functions at this point; it is orchestrating their behavior. This means that the logic for how each tool operates is predefined in the code, and the agent focuses on selecting the right tool for the job and providing the correct input to that tool.

Because agents can adapt as the loop progresses, they can dynamically decide which tool to use based on the current context and task requirements. This ability allows the agent to adjust its behavior as new information becomes available, making it more flexible and responsive to the user's input.

For example, if the user asks the agent to read the contents of a specific file, the agent will first use the `list_files` tool to identify the available files. Then, based on the result, it will determine whether to proceed with the `read_file` tool or respond with an error if the file does not exist. The agent evaluates each step iteratively, ensuring its actions are informed by the current state of the environment.

This orchestration process, driven by the **agent rules** and the tools available, showcases the power of combining pre-defined functions with adaptive decision-making. By allowing the agent to focus on **what to do** rather than **how to do it**, we create a system that leverages the LLM for high-level reasoning while relying on well-defined code for execution.

This separation of reasoning and execution is what makes the agent loop so powerful—it creates a modular, extensible framework that can handle increasingly complex tasks without rewriting the underlying tools.

Additionally, the agent loop eliminates much of the "glue code" traditionally required to tie these fundamental functions together. Instead of handcoding workflows, the agent dynamically decides the sequence of actions needed to achieve a task, effectively realizing a program on top of its components. This dynamic nature enables the agent to combine its tools in ways that would typically require custom logic, making it far more versatile and capable of addressing a broader range of use cases without additional development overhead.

Example in practice:

If the user asks, "What files are here?", the agent rules guide the LLM to respond with something like:

```
{
  "tool_name": "list_files",
  "arg": {}
}
```

This response ensures the agent's next step is both predictable and executable within its predefined constraints.

How agent rules integrate with the loop:

The `agent_rules` are combined with the `memory` in **Step 1: Construct Prompt** to form the input for the LLM. This guarantees that the agent always has access to its instructions and tools at every iteration. We will discuss the memory in more detail later.

This step prepares the input for the LLM by combining the system rules and the memory of the agent's previous interactions. The goal is to give the LLM all the necessary context for generating the next action.

Example in practice:

If the user asks, "What files are in this directory?", the memory might look like this:

```
memory = [
    ("role": "user", "content": "What files are in this directory?"),
    ("role": "assistant", "content": "...action('list_files', 'list_files', 'arg': {'dir': '.'})"),
    ("role": "user", "content": "[{'file1.txt'}, {'file2.txt'}]"),
]
```

Adding `agent_rules` ensures the LLM understands what tools it can use to continue interacting.

Step 2: Generate Response

After constructing the prompt, the agent sends it to the LLM to receive a response. This response will define the next action for the agent to execute.

Code snippet:

```
response = generate_response(prompt)
```

Explanation:

The `generate_response` function uses the `LiteLLM` library to send the prompt to the LLM and

⌵ Expand

✓ Completed

Go to next item